
DeepProbLog: Neural Probabilistic Logic Programming

Robin Manhaeve
KU Leuven
robin.manhaeve@cs.kuleuven.be

Sebastijan Dumančić
KU Leuven
sebastijan.dumancic@cs.kuleuven.be

Angelika Kimmig
Cardiff University
KimmigA@cardiff.ac.uk

Thomas Demeester*
Ghent University - imec
thomas.demeester@ugent.be

Luc De Raedt*
KU Leuven
luc.deraedt@cs.kuleuven.be

Abstract

We introduce DeepProbLog, a probabilistic logic programming language that incorporates deep learning by means of neural predicates. We show how existing inference and learning techniques can be adapted for the new language. Our experiments demonstrate that DeepProbLog supports (i) both symbolic and sub-symbolic representations and inference, (ii) program induction, (iii) probabilistic (logic) programming, and (iv) (deep) learning from examples. To the best of our knowledge, this work is the first to propose a framework where general-purpose neural networks and expressive probabilistic-logical modeling and reasoning are integrated in a way that exploits the full expressiveness and strengths of both worlds and can be trained end-to-end based on examples.

1 Introduction

The integration of low-level perception with high-level reasoning is one of the oldest, and yet most current open challenges in the field of artificial intelligence. Today, low-level perception is typically handled by neural networks and deep learning, whereas high-level reasoning is typically addressed using logical and probabilistic representations and inference. While it is clear that there have been breakthroughs in deep learning, there has also been a lot of progress in the area of high-level reasoning. Indeed, today there exist approaches that tightly integrate logical and probabilistic reasoning with statistical learning; cf. the areas of statistical relational artificial intelligence [De Raedt et al., 2016, Getoor and Taskar, 2007] and probabilistic logic programming [De Raedt and Kimmig, 2015]. Recently, a number of researchers have revisited and modernized older ideas originating from the field of neural-symbolic integration [Garcez et al., 2012], searching for ways to combine the best of both worlds [Bošnjak et al., 2017, Rocktäschel and Riedel, 2017, Cohen et al., 2018, Santoro et al., 2017], for example, by designing neural architectures representing differentiable counterparts of symbolic operations in classical reasoning tools. Yet, joining the full flexibility of high-level probabilistic reasoning with the representational power of deep neural networks is still an open problem. This paper tackles this challenge from a different perspective. Instead of integrating reasoning capabilities into a complex neural network architecture, we proceed the other way round. We start

* joint last authors

from an existing probabilistic logic programming language, ProbLog [De Raedt et al., 2007], and extend it with the capability to process neural predicates. The idea is simple: in a probabilistic logic, atomic expressions of the form $q(t_1, \dots, t_n)$ (aka tuples in a relational database) have a probability p . Consequently, the output of neural network components can be encapsulated in the form of “neural” predicates as long as the output of the neural network on an atomic expression can be interpreted as a probability. This simple idea is appealing as it allows us to retain all the essential components of the ProbLog language: the semantics, the inference mechanism, as well as the implementation. The main challenge is in training the model based on examples. The input data consists of feature vectors at the input of the neural network components (e.g., images) together with other probabilistic facts and clauses in the logic program, whereas targets are only given at the output side of the probabilistic reasoner. However, the algebraic extension of ProbLog (based on semirings) [Kimmig et al., 2011] already supports automatic differentiation. As a result, we can back-propagate the gradient from the loss at the output through the neural predicates into the neural networks, which allows training the whole model through gradient-descent based optimization. We call the new language *DeepProbLog*.

Before going into further detail, the following example illustrates the possibilities of this approach (also see Section 6). Consider the predicate $\text{addition}(X, Y, Z)$, where X and Y are images of digits and Z is the natural number corresponding to the sum of these digits. After training, DeepProbLog allows us to make a probabilistic estimate on the validity of, e.g., the example $\text{addition}(\text{3}, \text{5}, 8)$. While such a predicate can be directly learned by a standard neural classifier, such a method would have a hard time taking into account background knowledge such as the definition of the addition of two *natural* numbers. In DeepProbLog such knowledge can easily be encoded in rules such as $\text{addition}(I_X, I_Y, N_Z) :- \text{digit}(I_X, N_X), \text{digit}(I_Y, N_Y), N_Z \text{ is } N_X + N_Y$ (with *is* the standard operator of logic programming to evaluate arithmetic expressions). All that needs to be learned in this case is the neural predicate *digit* which maps an image of a digit I_D to the corresponding natural number N_D . The learned network can then be reused for arbitrary tasks involving digits. Our experiments show that this leads not only to new capabilities but also to significant performance improvements. An important advantage of this approach compared to standard image classification settings is that it can be extended to multi-digit numbers without additional training. We note that the single digit classifier (i.e., the neural predicate) is not explicitly trained by itself: its output can be considered a latent representation, as we only use training data with pairwise sums of digits.

To summarize, we introduce DeepProbLog which has a unique set of features: (i) it is a programming language that supports neural networks and machine learning, and it has a well-defined semantics (as an extension of Prolog, it is Turing equivalent); (ii) it integrates logical reasoning with neural networks; so both symbolic and subsymbolic representations and inference; (iii) it integrates probabilistic modeling, programming and reasoning with neural networks (as DeepProbLog extends the probabilistic programming language ProbLog, which can be regarded as a very expressive directed graphical modeling language [De Raedt et al., 2016]); (iv) it can be used to learn a wide range of probabilistic logical neural models from examples, including inductive programming. The code is available at <https://bitbucket.org/problog/deepproblog>.

2 Logic programming concepts

We briefly summarize basic logic programming concepts. Atoms are expressions of the form $q(t_1, \dots, t_n)$ where q is a predicate (of arity n) and the t_i are terms. A literal is an atom or the negation $\neg q(t_1, \dots, t_n)$ of an atom. A term t is either a constant c , a variable V , or a structured term of the form $f(u_1, \dots, u_k)$ where f is a functor. We will follow the Prolog convention and let constants start with a lower case character and variables with an upper case. A substitution $\theta = \{V_1 = t_1, \dots, V_n = t_n\}$ is an assignment of terms t_i to variables V_i . When applying a substitution θ to an expression e we simultaneously replace all occurrences of V_i by t_i and denote the resulting expression as $e\theta$. Expressions that do not contain any variables are called ground. A rule is an expression of the form $h :- b_1, \dots, b_n$ where h is an atom and the b_i are literals. The meaning of such a rule is that h holds whenever the conjunction of the b_i holds. Thus $:-$ represents logical implication ($\leftarrow$), and the comma $(,)$ represents conjunction ($\wedge$). Rules with an empty body $n = 0$ are called facts.

3 Introducing DeepProbLog

We now recall the basics of probabilistic logic programming using ProbLog, illustrate it using the well-known burglary alarm example, and then introduce our new language DeepProbLog.

A ProbLog program consists of (i) a set of ground probabilistic facts $\mathcal{F}$ of the form $p :: f$ where p is a probability and f a ground atom and (ii) a set of rules $\mathcal{R}$. For instance, the following ProbLog program models a variant of the well-known alarm Bayesian network:

```

0.1 :: burglary.    0.5 :: hears_alarm(mary).    alarm :- earthquake.
0.2 :: earthquake. 0.4 :: hears_alarm(john).      alarm :- burglary.
                                     calls(X) :- alarm, hears_alarm(X).

```

(1)

Each probabilistic fact corresponds to an *independent Boolean random variable* that is true with probability p and false with probability $1 - p$. Every subset $F \subseteq \mathcal{F}$ defines a possible world $w_F = F \cup \{f\theta \mid \mathcal{R} \cup F \models f\theta \text{ and } f\theta \text{ is ground}\}$. So w_F contains F and all ground atoms that are logically entailed by F and the set of rules $\mathcal{R}$, e.g.,

$$w_{\{\text{burglary, hears_alarm(mary)}\}} = \{\text{burglary, hears_alarm(mary)}\} \cup \{\text{alarm, calls(mary)}\}$$

The probability $P(w_F)$ of such a possible world w_F is given by the product of the probabilities of the truth values of the probabilistic facts, $P(w_F) = \prod_{f_i \in F} p_i \prod_{f_i \in \mathcal{F} \setminus F} (1 - p_i)$. For instance,

$$P(w_{\{\text{burglary, hears_alarm(mary)}\}}) = 0.1 \times 0.5 \times (1 - 0.2) \times (1 - 0.4) = 0.024$$

The probability of a ground fact q , also called *success probability* of q , is then defined as the sum of the probabilities of all worlds containing q , i.e., $P(q) = \sum_{F \subseteq \mathcal{F}: q \in w_F} P(w_F)$.

One convenient extension that is nothing else than syntactic sugar are the annotated disjunctions. An annotated disjunction (AD) is an expression of the form $p_1 :: h_1; \dots; p_n :: h_n :- b_1, \dots, b_m$, where the p_i are probabilities so that $\sum p_i = 1$, and h_i and b_j are atoms. The meaning of an AD is that whenever all b_i hold, h_j will be true with probability p_j , with all other h_i false (unless other parts of the program make them true). This is convenient to model choices between different categorical variables, e.g. different severities of the earthquake:

```

0.4 :: earthquake(none); 0.4 :: earthquake(mild); 0.2 :: earthquake(severe).

```

ProbLog programs with annotated disjunctions can be transformed into equivalent ProbLog programs without annotated disjunctions (cf. De Raedt and Kimmig [2015]).

A **DeepProbLog** program is a ProbLog program that is extended with (iii) a set of ground neural ADs (nADs) of the form

$$nn(m_q, \vec{t}, \vec{u}) :: q(\vec{t}, u_1); \dots; q(\vec{t}, u_n) :- b_1, \dots, b_m$$

where the b_i are atoms, $\vec{t} = t_1, \dots, t_k$ is a vector of ground terms representing the inputs of the neural network for predicate q , u_1 to u_n are the possible output values of the neural network. We use the notation nn for ‘neural network’ to indicate that this is a nAD. m_q is the identifier of a neural network model that specifies a probability distribution over its output values $\vec{u}$, given input $\vec{t}$. That is, from the perspective of the probabilistic logic program, an nAD realizes a regular AD $p_1 :: q(\vec{t}, u_1); \dots; p_n :: q(\vec{t}, u_n) :- b_1, \dots, b_m$, and DeepProbLog thus directly inherits its semantics, and to a large extent also its inference, from ProbLog. For instance, in the MNIST addition example, we would specify the nAD

$$nn(m_{\text{digit}}, \text{img}, [0, \dots, 9]) :: \text{digit}(\text{img}, 0); \dots; \text{digit}(\text{img}, 9).$$

where m_{digit} is a network that probabilistically classifies MNIST digits. The neural network could take any shape, e.g., a convolutional network for image encoding, a recurrent network for sequence encoding, etc. However, its output layer, which feeds the corresponding neural predicate, needs to be normalized. In neural networks for multiclass classification, this is typically done by applying a softmax layer to real-valued output scores, a choice we also adopt in our experiments.